Web Development

CSE343
Lab 3

JavaScript

Why use JavaScript in Web?

- In the last lab, we made static webpages using HTML and made there styling using CSS. What if we want to introduce logic to our webpage or become more dynamic and interactive?
- JavaScript is the answer!
- By adding the <script> tag to your HTML code, you can define JavaScript functions that modify the configurations of elements present in your webpage.
- JavaScript is a scripting language used to make web pages interactive.
- Runs in the browser (client-side) and can also run on servers (Node.js).
- Used to manipulate HTML, handle user input, and update the page dynamically.

Example:

```
<script>
  console.log('Hello, World!');
</script>
```



JavaScript Basics

Variables and Constants

- `let` – declares a block-scoped variable.
- `const` – declares a constant that cannot be reassigned.
- `var` – older way; function-scoped (avoid in modern JS).

Example:

```
let name = 'Alice';
const pi = 3.14;
var age = 25;
console.log(name, pi, age);
```

Do you know the difference between var and let?

Datatypes and Operators

- Primitive types: Number, String, Boolean, Null, Undefined, Symbol, BigInt
- Operators: +, -, *, /, %, ==, ===, !=, !==, &&, ||

Example:

```
let x = 10, y = 5;
console.log(x + y); // 15
console.log(x === y); // false
```

Do you know the difference between == and ===?

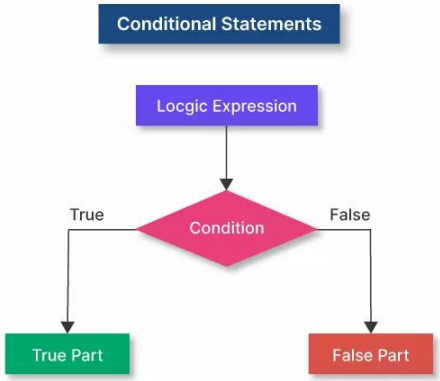
JavaScript Basics

Conditionals

- Used to execute code based on conditions.

Example:

```
let score = 85;
if (score >= 90) {
  console.log('A grade');
} else if (score >= 75) {
  console.log('B grade');
} else {
  console.log('C grade');
}
```



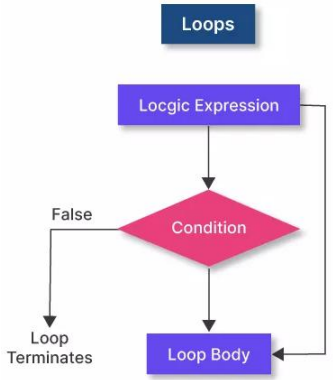
Also: switch statements for multiple cases.
Do you know how to write single line if conditions?

Loops

- Repeat code blocks until a condition is met.

```
for (let i = 0; i < 5; i++) {
  console.log('Count:', i);
}

let j = 0;
while (j < 3) {
  console.log('Hello');
  j++;
}
```



JavaScript Basics

Arrays

- Ordered lists of items accessed by index (start from 0).

let fruits = ['apple', 'banana', 'cherry'];

🔹 **Common Array Methods**

Method	Example	Description
`.push()`	`fruits.push("grape")`	Add to end
`.pop()`	`fruits.pop()`	Remove last item
`.shift()`	`fruits.shift()`	Remove first item
`.unshift()`	`fruits.unshift("kiwi")`	Add to start
`.length`	`fruits.length`	Count items
`.forEach()`	`fruits.forEach(f => console.log(f))`	Loop through items
`.map()`	`let caps = fruits.map(f => f.toUpperCase())`	Transform array
`.filter()`	`let short = fruits.filter(f => f.length < 6)`	Keep only matching items

Objects

- Store key–value pairs to represent structured data.

let person = { name: 'Alice', age: 25, city: 'London' };

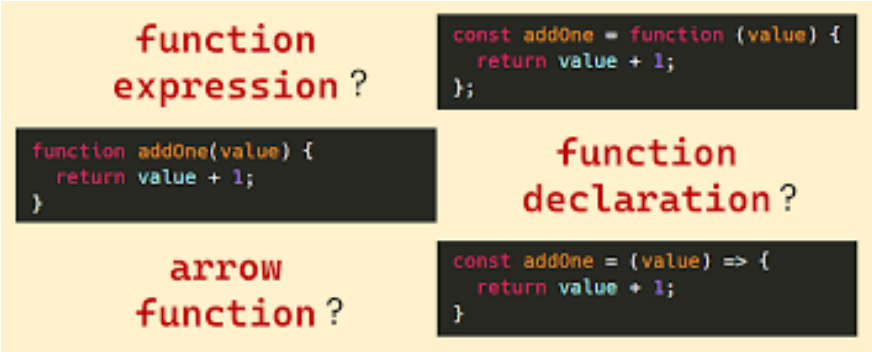
- Access via person.name or person['age'].
- Add or modify: person.country = 'UK';
- Loop: for (let key in person) { console.log(key, person[key]); }

Use Arrays for lists; Objects for grouped properties.

Example: let students = [{name:'John', grade:90}, {name:'Sara', grade:85}];

Functions Theory

- Functions are reusable blocks of code that perform specific tasks. They help make programs modular, readable, and maintainable.
- Declaration forms:
  - function declaration: Allows function hoisting (using before declaration)
  - function expression: assigned to variables, not hoisted.
  - arrow functions: short syntax, no own 'this', ideal for callbacks
- Scope determines where are variables seen and who has access to modify them: local vs global.
  - Local Scope – variables inside a function are private to it
  - Global Scope – accessible everywhere (avoid polluting global namespace by minimizing the usage of global scope as much as possible)
- Closures: functions remember variables from their creation scope even after it ends.
  - Has a private memory stack for every instance call for a function.
- Pure functions :
  - Always return the same output for the same input.
  - Have no side effects (don’t modify external state).
  - Easier to test, debug, and reason about.



Functions Example

- JS (snippets):

```
// declaration
function add(a, b) { return a + b; }
// expression
const sub = function(a, b) { return a - b; };
// arrow
const mul = (a, b) => a * b;
```

- Closure example:

```
function counter() { let c = 0; return () => ++c; }
const next = counter(); console.log(next()); // 1
```

Functions Example

```
<script>
  function addNumbers(a, b) {
    return a + b;
  }

  console.log(addNumbers(5, 7));
function counter() { let c = 0; return () => ++c; }
const next = counter(); console.log(next()); // 1
console.log(next());
console.log(next());

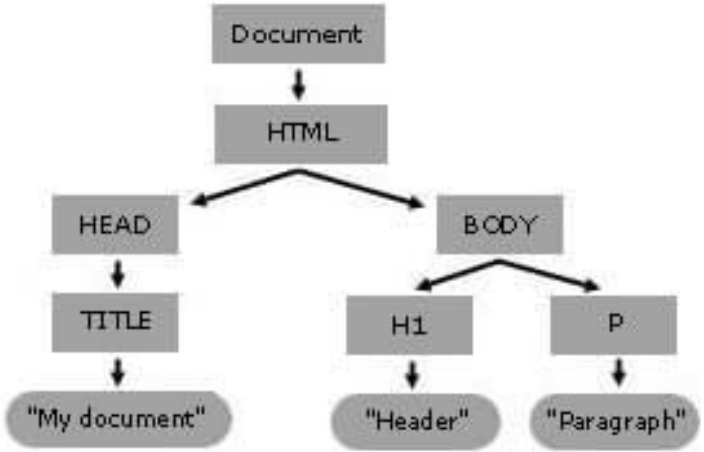
const next1 = counter();
console.log(next1());
console.log(next1());
</script>
```

- Can you explain why the output is like this?

```
12
1
2
3
1
2
```

DOM Theory

- The Document Object Model (DOM) is the browser's in-memory representation of an HTML document.
- The DOM is a tree of nodes: document -> element nodes -> text nodes.
- You can read and modify nodes via JavaScript (properties, attributes, methods).
- Common APIs: document.getElementById, querySelector, createElement, appendChild, textContent, innerHTML.



Onclick attribute – provide JS Function to run

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hello JavaScript</title>
</head>
<body>
  <h1>My First JavaScript Page</h1>
  <p id="message">Welcome!</p>
  <button onclick="changeMessage()">Click Me</button>

  <script>
    function changeMessage() {
      document.getElementById("message").textContent = "Hello from JavaScript!";
    }
  </script>
</body>
</html>
```





- Document Object -> P Object -> Text Key changed value from Welcome! To Hello from JavaScript

DOM Example: Access & Update – Predict Code Output

```
HTML (snippet inside <body> tag):
<p id='greeting'>Welcome</p>
<button id='sayHi'>Say Hi</button>

JS (snippet inside <script>):
const btn = document.getElementById('sayHi');
btn.addEventListener('click', () => {
  document.getElementById('greeting').textContent = 'Hello from JS!';
});
```

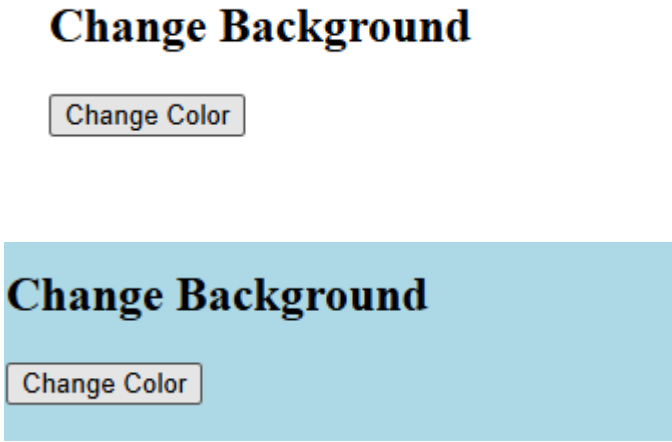
- Explanation: getElementById returns a node; textContent changes visible text without parsing HTML.
- Notice the use of **EventListener** instead of onclick attribute (We will discuss this further)

CSS Styling in JS

```html
<h2>Change Background</h2>
<button id="colorBtn">Change Color</button>

<script>
  const button = document.getElementById("colorBtn");

  button.addEventListener("click", () => {
    document.body.style.backgroundColor =
      document.body.style.backgroundColor === "lightblue" ? "white" : "lightblue";
  });
</script>
```



- As any object in the DOM model, <style> can be modified in JavaScript by accessing the required style attribute.
- The Inline If condition works as follows → Condition Check ? True output : False Output ;

DOM Example: Create Elements – Predict Code Output

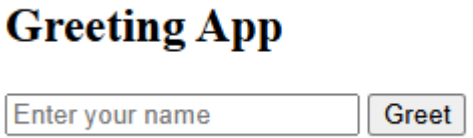
- JS (snippet):
- `const list = document.getElementById('todo');`
- `const li = document.createElement('li');`
- `li.textContent = 'New item';`
- `list.appendChild(li);`

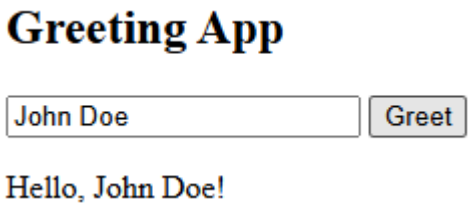
- Explanation: createElement + appendChild adds new nodes without rebuilding the page.
- Note that JS can add HTML elements to your webpage based on the logic found in the JS code without you writing the HTML Tags etc..

User Inputs and Template Literals

```html
<h2>Greeting App</h2>
<input type="text" id="nameInput" placeholder="Enter your name">
<button id="greetBtn">Greet</button>
<p id="output"></p>

<script>
  document.getElementById("greetBtn").addEventListener("click", () => {
    const name = document.getElementById("nameInput").value;
    document.getElementById("output").textContent = `Hello, ${name}!`;
  });
</script>
```

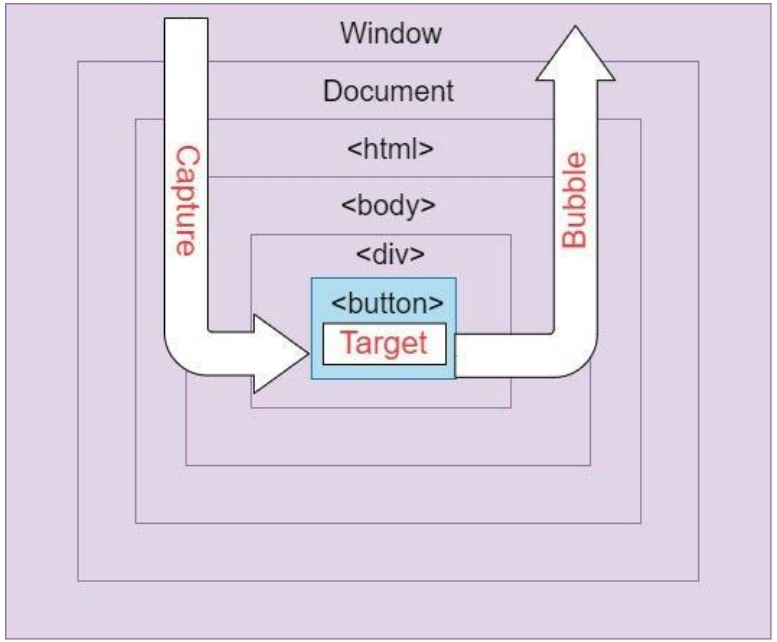




- We first accessed the value inside the object input with ID=nameInput.
- Notice how the const name is used in the <p> output (Template Literals allow for having placeholders in the code that are later filled by the actual value based on the interaction with the user)

Events Theory

- Events represent user or browser actions: click, input, keydown, submit, load, etc.
- Two important concepts: event listener registration and event propagation (capturing/bubbling).
- Use addEventListener for adding handlers; avoid inline handlers (ex:onclick) for separation of concerns between HTML and JS.
- Event object provides details: event.target, event.currentTarget, event.preventDefault(), event.stopPropagation(). Event is passed to routine that was listening to the event.



Event Propagation – Bubbling






```html
<h2>Event Bubbling in Action</h2>

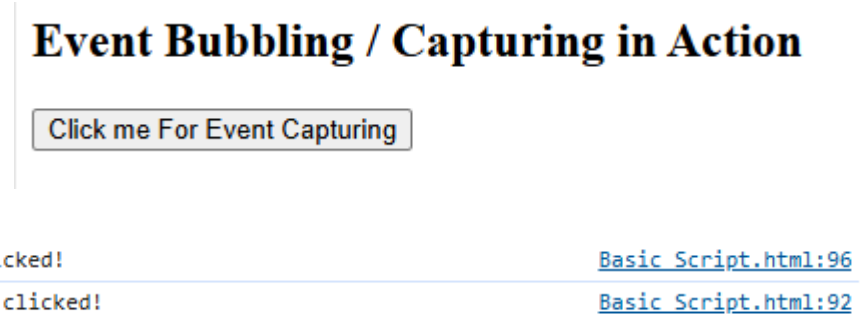
<div id="outer">
  <button id="inner">Click me For Bubbling</button>
</div>

<script>
  document.getElementById("inner").addEventListener("click", () => {
  console.log("Button clicked!");
});

document.getElementById("outer").addEventListener("click", () => {
  console.log("Div clicked!");
});
</script>
```

Event Propagation – Capturing

- Notice how the code on line 96 is avtivated first before the code on line 92 !!
- addEventListener(event type, Listener Routine, Use Capture)
- The default mode is bubbling. Only use capturing when you want the parent to handle the event (ex: Delegation)




```
<script>
  document.getElementById("inner").addEventListener("click", () => {
  console.log("Button clicked!");
});

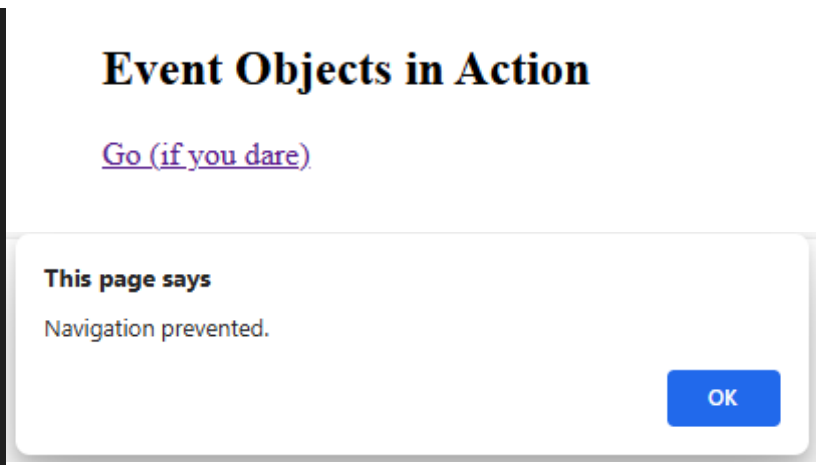
document.getElementById("outer").addEventListener("click", () => {
  console.log("Div clicked!");
},true);
</script>
<!-- (,true) here activates the parent in capturing mode to handle events before the target that activated the event -->
```

Events Example: Click & preventDefault

```html
<h2>Event Objects in Action</h2>

<a href='https://youtube.com' id='link'>Go (if you dare)</a>

<script>
  const link = document.getElementById('link');
link.addEventListener('click', (event) => {
  event.preventDefault(); // stops navigation
  alert('Navigation prevented.');
});
</script>
```



- Explanation: preventDefault cancels default browser action for that event.

Common Event Objects Properties

Property	Description	Example
`e.type`	The type of event	`"click"`, `"input"`, `"keydown"`
`e.target`	The element that triggered the event	If you click a `<button>` inside a `<div>`, then `event.target` → `<button>`
`e.currentTarget`	The element the listener is attached to	If you click a `<button>` inside a `<div>`, and listen in `<div>`, then `event.currentTarget` → `<div>`
`e.clientX`, `e.clientY`	Mouse position (for mouse events)	`120`, `350`
`e.key`	Key pressed (for keyboard events)	`"Enter"`, `"a"`
`e.preventDefault()`	Stops the default browser action	e.g., stops a link from navigating
`e.stopPropagation()`	Stops the event from bubbling	useful for nested elements

Events Example: Delegation – Predict Code Output

- Use delegation to attach one handler to a parent instead of many children.

```
HTML (snippet)
<ul id="list">
    <li>Apple</li>
    <li>Banana</li>
    <li>Cherry</li>
    <li>Orange</li>
    <li>Grapes</li>
  </ul>

JS (snippet):
document.getElementById('list').addEventListener('click', (e) => {
  if (e.target && e.target.nodeName === 'LI') {
    console.log('Clicked item:', e.target.textContent);
  } });
```

Best Practices & Debugging

- Use addEventListener instead of inline handlers; remove listeners when not needed.
- Prefer textContent over innerHTML unless you need HTML parsing.
- Use console.log and browser devtools (Elements, Console, Network, Performance).
- Keep functions small and testable; use descriptive names and comments.

```html
<!doctype html>
<html>
  <body>
    <h1 id="title">Welcome</h1>
    <input id="name" placeholder="Your name" />
    <button id="greet">Greet</button>
    <ul id="list"></ul>

    <script>
      // Function to greet the user and add their name to the list
      function addName() {
        const input = document.getElementById('name');
        const name = input.value.trim();

        if (!name) return; // ignore empty input

        document.getElementById('title').textContent = `Hello, ${name}!`;

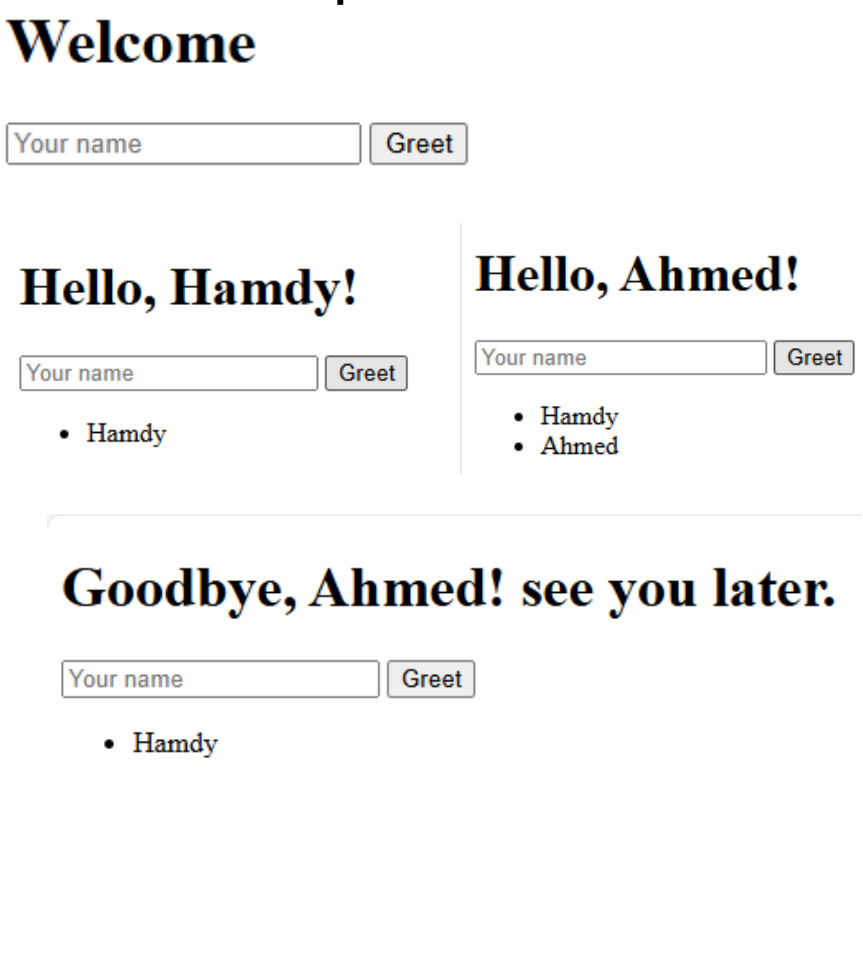
        const li = document.createElement('li');
        li.textContent = name;
        document.getElementById('list').appendChild(li);

        input.value = ''; // clear input box
      }

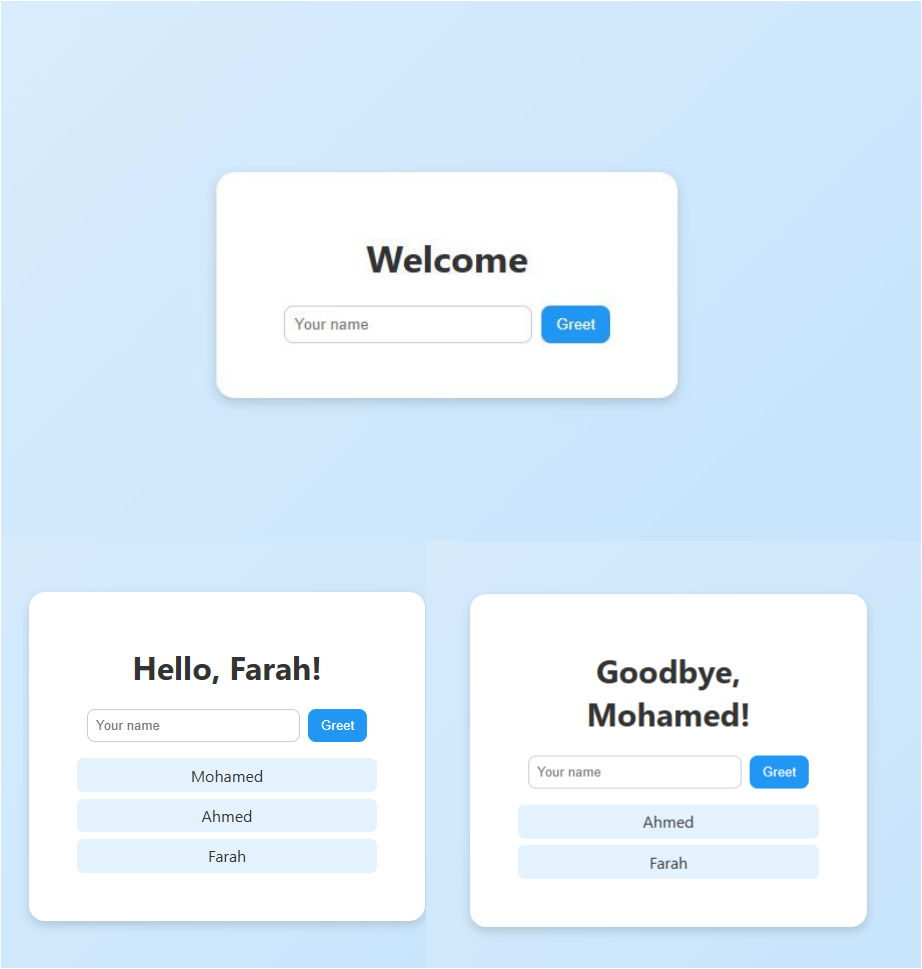
      // Function to say goodbye when a name is clicked
      function removeName(e) {
        if (e.target.nodeName === 'LI') {
          const name = e.target.textContent;
          document.getElementById('title').textContent = `Goodbye, ${name}! see you later.`;
          e.target.remove(); // remove the clicked <li>
        }
      }

      // Attach event listeners
      document.getElementById('greet').addEventListener('click', addName);
      document.getElementById('list').addEventListener('click', removeName);
    </script>
  </body>
</html>
```

List Example: Reservations



Lab Demo


Welcome
Your name
Greet
Hello, Farah!
Your name
Greet
Mohamed
Ahmed
Farah
Goodbye, Mohamed!
Your name
Greet
Ahmed
Farah

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Greet & Goodbye Demo</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <div class="container">
      <h1 id="title">Welcome</h1>
      <div class="input-area">
        <input id="name" placeholder="Your name" />
        <button id="greet">Greet</button>
      </div>
      <ul id="list"></ul>
    </div>

    <script src="script.js"></script>
  </body>
</html>
```

External CSS as seen in last lab

Path to External JS Script added in src attribute for <script>

Lab Task

My Interactive Calculator Webpage

- Prepare the .HTML File, .CSS File, and .JS file to make a webpage calculator for basic mathematical operations.
- Be creative in CSS and Efficient in JS, HTML.
- Hints: Break tasks into DOM (select nodes), Event (listen), Logic (functions).

